OpenClaw Connecting to Robot MCP Service

OpenClaw Connecting to Robot MCP Service

1. Course Content
   - Course Overview
   - Learning Objectives
2. Preparation
3. MCP Protocol Introduction
4. OpenClaw Robot Control Communication Architecture
5. Demo Cases
   - Conversational Control Examples
6. Core Source Code Analysis
   - 6.1 🖥️ MCP Service Entry (`mcp_service.py`)
   - 6.2 🛠️ MCP Tools Registration (`mcp_action.py`)
   - 6.3 💬 MCP Response Message Format (`mcp_message.py`)

1. Course Content

Course Overview

This lesson explains how to use the OpenClaw intelligent agent framework to achieve autonomous intelligent control of the A1Pro Ackermann mobile robot and robotic arm through the **MCP (Model Context Protocol)** interface with large language models (LLM).

Learning Objectives

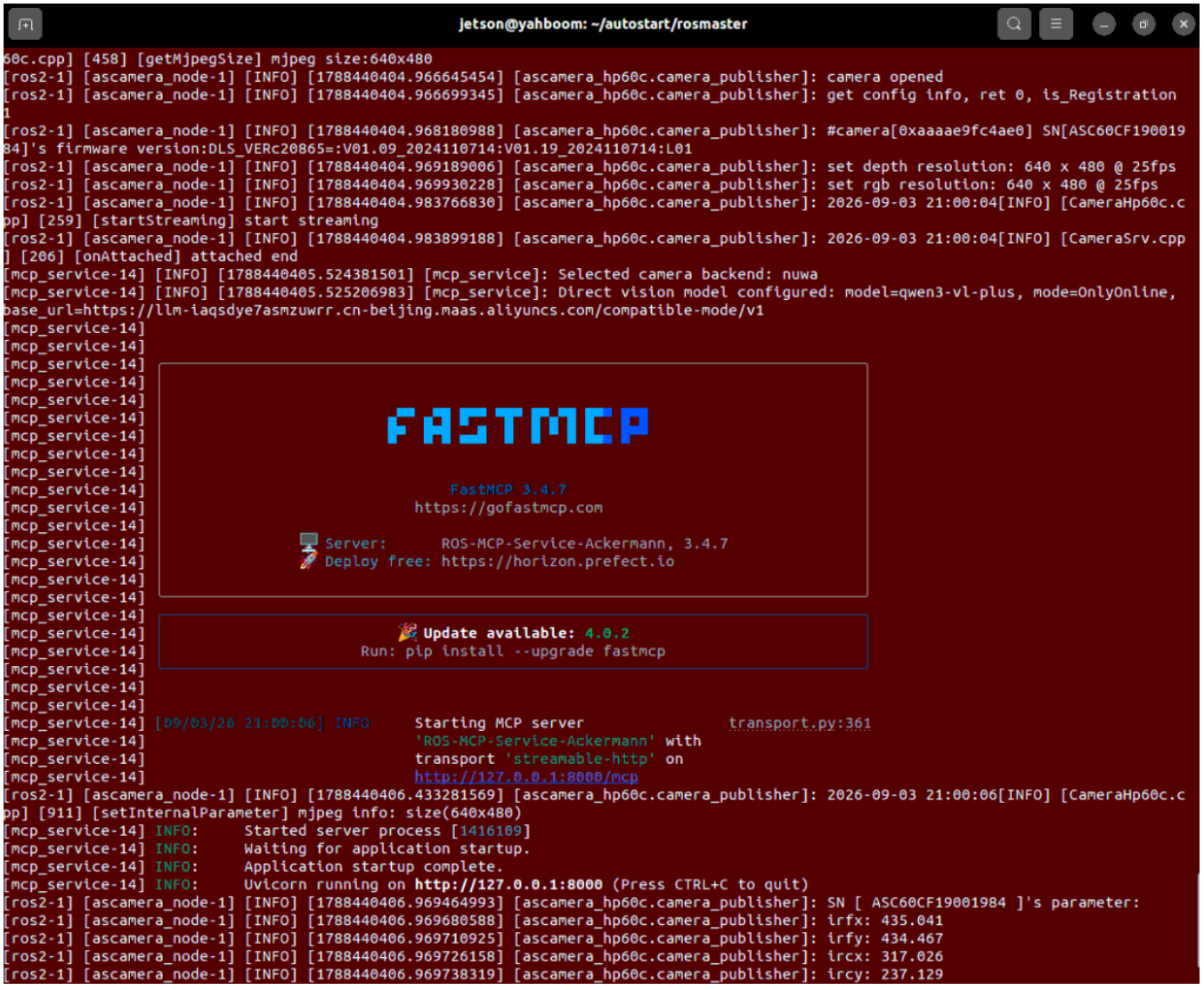
1. Understand the basic principles of the MCP protocol and its role in robotic systems
2. Master how to start the robot-side MCP service and its operating architecture
3. Master how to schedule Ackermann chassis and robotic arm collaborative operations through OpenClaw conversational interaction (WebChat / Natural Language)

---

2. Preparation

Execute the following startup command on the robot terminal:

```
ros2 launch dofbot_pro_bringup mcp_service.launch.py
```

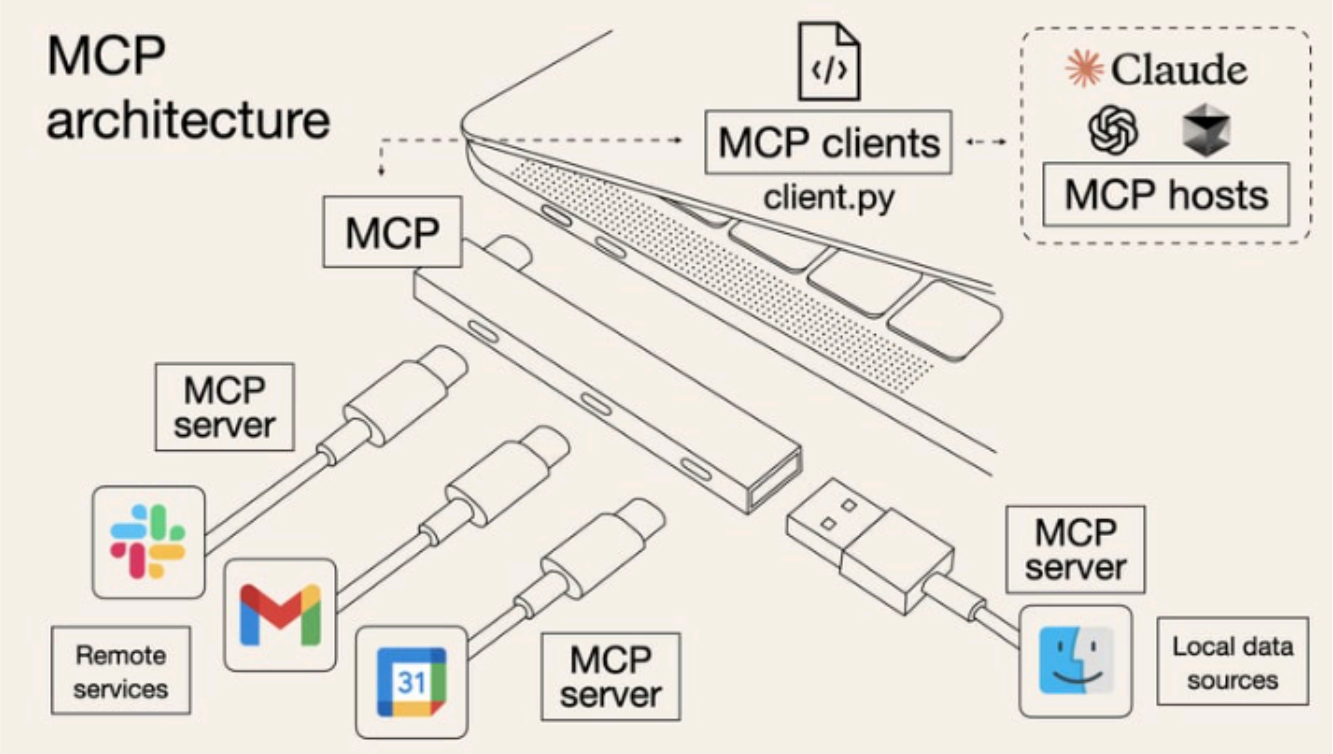


> **Note**
>
> This Launch script will automatically detect the currently equipped camera type (USB monocular camera or Nuwa 3D depth camera) and one-click pull up the chassis and robotic arm communication drivers, camera driver, kinematics resolution node, point cloud/vision recognition service, and the FastMCP server running in the background listening on port `8000`.

---

3. MCP Protocol Introduction

- **MCP (Model Context Protocol)** is an open standard communication protocol designed to seamlessly connect large language models with external tools (Tools), data sources (Resources), and real hardware services.
- In this system, OpenClaw acts as the upper-level intelligent brain, converting user natural language instructions into structured Tool Calls through the MCP protocol.
- The `ackermann_mcp` server deployed on the robot receives tool calls and distributes them to underlying ROS 2 topic publishing, service requests, or action goals, driving the Ackermann steering chassis, 6-DOF robotic arm, and vision algorithms to execute physical operations, then formats the execution results and returns them to the large model.



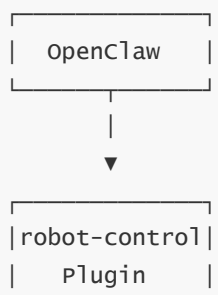
Through standardized MCP protocol, robots can be plug-and-play integrated with any MCP-supported AI Agent or large model client (such as Claude Desktop, Cursor, OpenClaw, etc.) without repeatedly writing private control protocols for each model.

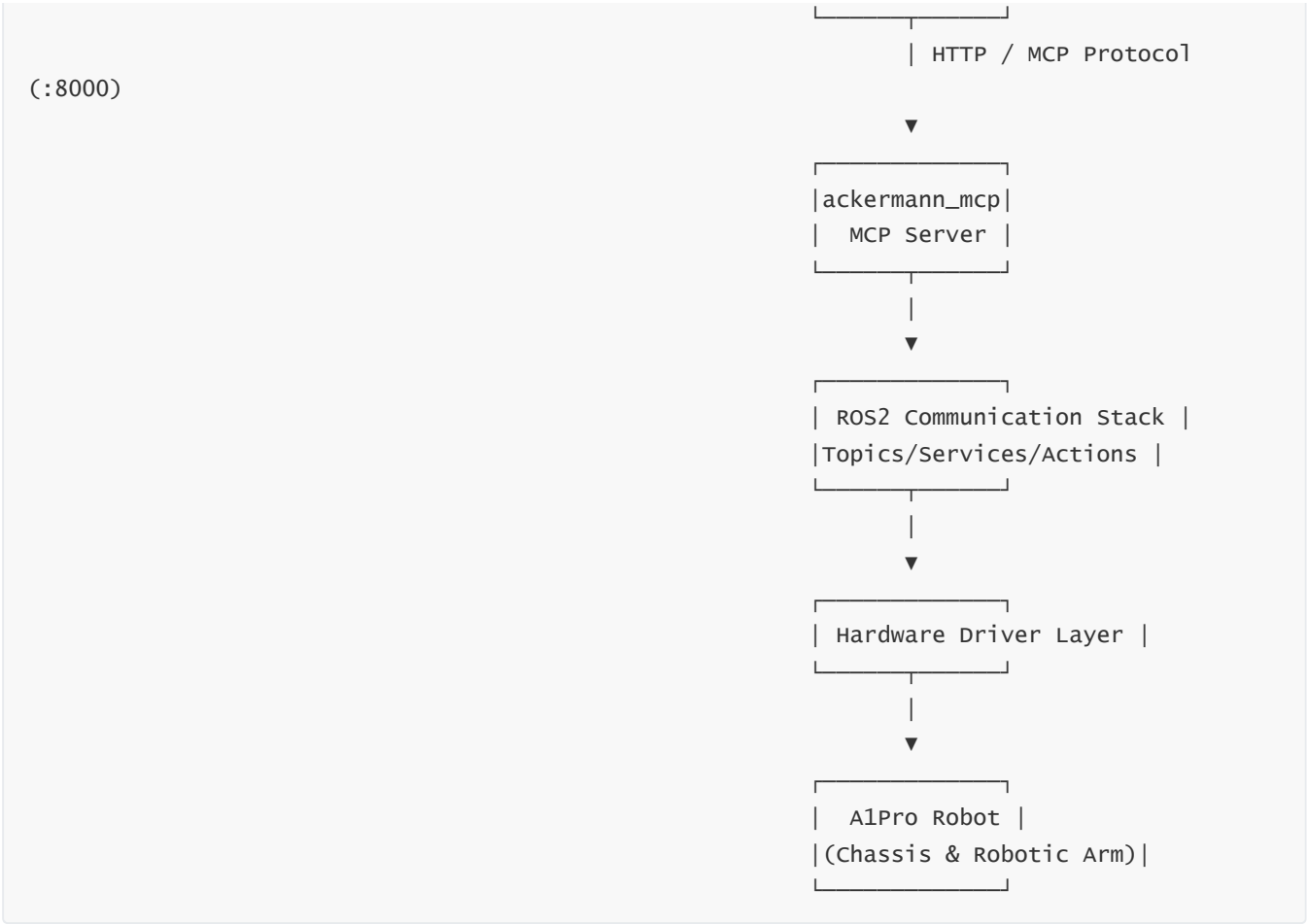
---

4. OpenClaw Robot Control Communication Architecture

The communication data flow of the entire system is divided into five core layers:

1. **User Interaction Layer**: Users input natural language through WebChat, Feishu, WeChat, voice (ASR) and other entry points;
2. **AI Decision Layer**: OpenClaw agent calls the large model to reason about user intent and decides which MCP Tools and actual parameters need to be executed from the registered tool library;
3. **Plugin Bridge Layer**: OpenClaw's built-in `robot-control` plugin establishes a Session with the robot-side MCP server through HTTP/SSE protocol and initiates Tool calls;
4. **MCP Service Layer (`ackermann_mcp`)**: Running on the robot locally based on the FastMCP framework (port 8000), managing all action controllers (`ActionController`) of the robot;
5. **Hardware Execution Layer**: Through ROS 2 nodes and underlying microcontroller drivers, controlling the Ackermann steering servo, power motor, and robotic arm bus servo, forming a complete execution closed loop.





5. Demo Cases

Open the OpenClaw WebChat interface in a PC browser on the same local area network as the robot and log in.

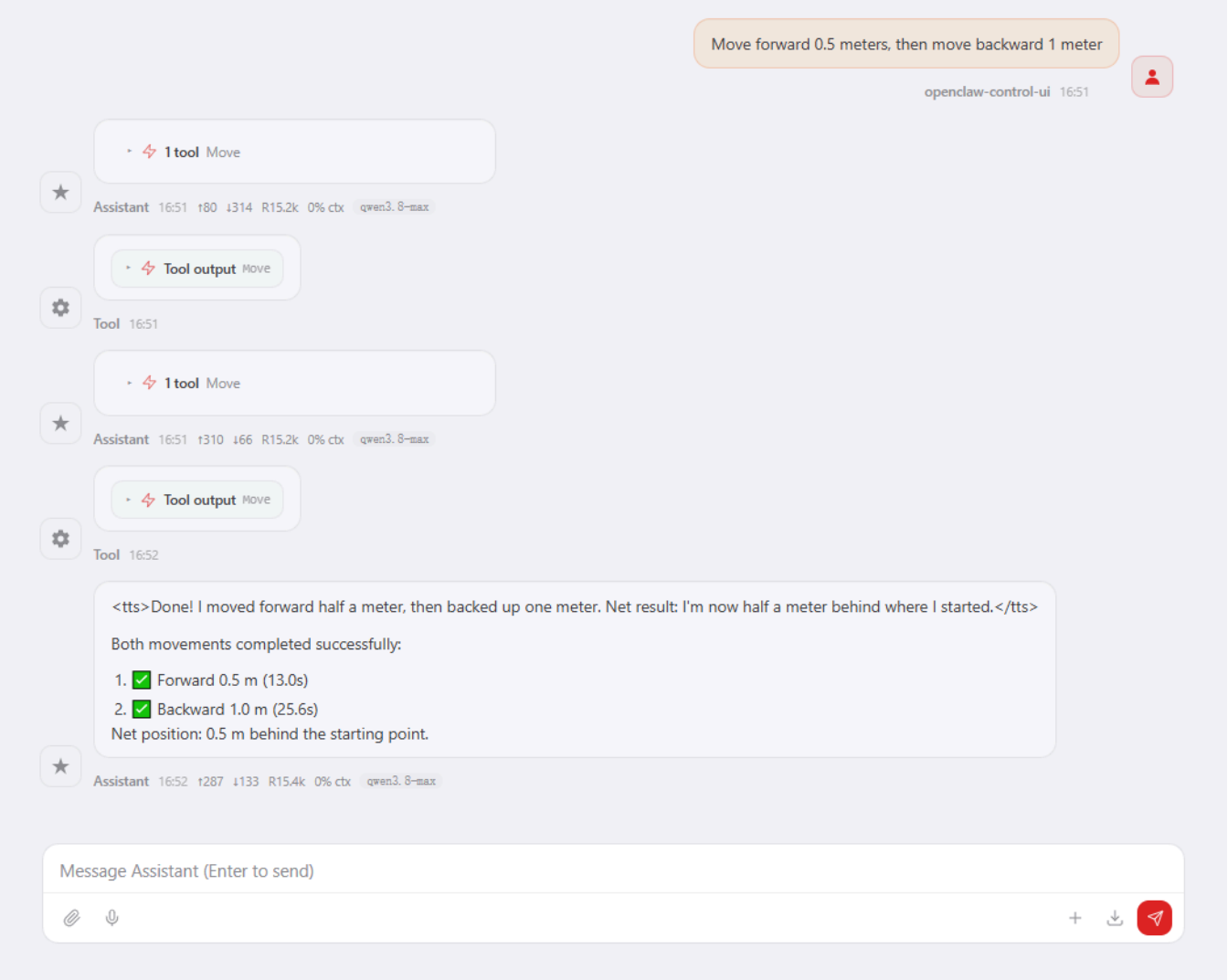
> **Note**
>
> Here we demonstrate using WebChat conversational interaction as an example. You can also use voice, terminal TUI, or any other OpenClaw access channel.

Conversational Control Examples

Send natural language instructions to OpenClaw:

- "Move forward 0.5 meters, then move backward 1 meter"
- "Reset the robotic arm pose and observe the current environment"

OpenClaw automatically plans and continuously calls MCP tools such as `Move`, `InitArmPose`, `SeeWhat`:



6. Core Source Code Analysis

6.1 💻 MCP Service Entry (`mcp_service.py`)

Source file: `dofbot_pro_ws/src/ackermann_mcp/ackermann_mcp/mcp_service.py`

`mcp_service.py` is the main entry point for the robot-side MCP protocol service, primarily completing the following lifecycle management:

1. **FastMCP Service Instantiation**: Create a FastMCP service object named `"ROS-MCP-Service-Ackermann"`;
2. **Initialize ROS 2 Controller**: Call `create_controller()` to create a background action controller and start `start_ros_spin()` to start the asynchronous event loop;
3. **Register Robot Tool Set**: Call `register_actions()` to expose all capabilities such as chassis movement, robotic arm joint control, visual tracking, and autonomous grasping as standard MCP Tools;
4. **Bind Network Service Port**: Listen on local `0.0.0.0:8000` port, providing long connection service in `streamable-http` mode.

```
# mcp_service.py Core Startup Logic
from fastmcp import FastMCP
```

```
from .action_controller import create_controller, shutdown_ros, start_ros_spin
from .mcp_tools.mcp_action import register_actions
from .runtime.process_env import configure_fastdds_realloc_history

def main(args=None):
    configure_fastdds_realloc_history()
    try:
        # 1. Instantiate FastMCP
        mcp_service = FastMCP(name="ROS-MCP-Service-Ackermann")
        # 2. Create ROS 2 action controller
        action_controller = create_controller()
        start_ros_spin()
        # 3. Register complete robot control tools
        register_actions(mcp=mcp_service, action_controller=action_controller)
        # 4. Start service listening on port 8000
        mcp_service.run(
            host=action_controller.MCP_HOST,
            port=action_controller.MCP_PORT,
            transport=action_controller.MCP_TRANSPORT
        )
    finally:
        shutdown_ros()
```

6.2 🛠️ MCP Tools Registration (`mcp_action.py`)

Source file: `dofbot_pro_ws/src/ackermann_mcp/ackermann_mcp/mcp_tools/mcp_action.py`

All tools oriented toward the large model are centrally declared and registered in this file using the `@mcp.tool()` decorator. The tools are equipped with detailed semantic descriptions (Description) and parameter constraints (Field) to help the large model precisely understand calling rules.

Taking **Chassis Movement (`Move`)** and **6-DOF Robotic Arm Control (`ControlSixArmJoint`)** as examples:

```
# mcp_action.py —— Chassis Movement Tool Registration
@mcp.tool(
    description='''Control the robot's chassis to move a specified distance. '''
                '''Supported directions: 'forward', 'backward', 'left' (turn and drive),
'right' (turn and drive). '''
                '''The chassis will execute motion based on Ackermann steering geometry.''',
    timeout=30.0
)
def Move(
    distance: Annotated[float, Field(description="Distance to move in meters (must be
positive)")],
    direction: Annotated[str, Field(description="Movement direction: 'forward', 'backward',
'left', or 'right'")] = 'forward'
) -> common_response:
    res = action_controller.move_distance(distance=distance, direction=direction)
    return common_response(execution_result="success" if res[0] else "failed",
reason=res[1])

# mcp_action.py —— 6-DOF Robotic Arm Multi-Joint Coordination Tool Registration
```

```
@mcp.tool(
    description="Simultaneously control 6 joints of the robotic arm to set precise angles.",
    timeout=15.0
)
def ControlSixArmJoint(
    arm_joint_1: Annotated[int, Field(ge=0, le=180, description="Base rotation angle [0,
180]")],
    arm_joint_2: Annotated[int, Field(ge=0, le=180, description="Shoulder pitch angle [0,
180]")],
    arm_joint_3: Annotated[int, Field(ge=0, le=180, description="Elbow pitch angle [0,
180]")],
    arm_joint_4: Annotated[int, Field(ge=0, le=180, description="Wrist pitch angle [0,
180]")],
    arm_joint_5: Annotated[int, Field(ge=0, le=180, description="Wrist roll angle [0,
180]")],
    arm_joint_6: Annotated[int, Field(ge=0, le=180, description="Gripper angle [0, 180]")],
    runtime: Annotated[int, Field(default=1500, description="Movement duration in ms")] =
1500
) -> common_response:
    res = action_controller.control_six_arm(
        arm_joint_1, arm_joint_2, arm_joint_3, arm_joint_4, arm_joint_5, arm_joint_6,
runtime
    )
    return common_response(execution_result="success" if res[0] else "failed",
reason=res[1])
```

6.3 💬 MCP Response Message Format (`mcp_message.py`)

Source file: `dofbot_pro_ws/src/ackermann_mcp/ackermann_mcp/message/mcp_message.py`

All tool execution feedback is serialized and encapsulated using lightweight Python Dataclasses, uniformly returning a struct containing `execution_result` (`"success"` or `"failed"`) and `reason` detailed cause, allowing the large model to quickly reflect and adaptively plan based on results:

```
# mcp_message.py -- Common Response Data Structure
from dataclasses import dataclass

@dataclass
class common_response:
    execution_result: str   # Execution status: "success" or "failed"
    reason: str             # Detailed execution log or failure diagnosis reason

    def __post_init__(self):
        # Fallback to ensure reason is not empty, preventing empty text from blocking
downstream Agent decision flow
        if (self.reason is None or self.reason == "") and self.execution_result is not None:
            self.reason = str(self.execution_result)
```

In addition to common responses, the module also encapsulates data structures such as `ArmJointAngles` (robotic arm joint readings), `EndEffectorPose` (end-effector spatial 6DoF pose), `BoxLocalization` (visual target bounding box), and `waste_recognition_result` (waste classification detection), providing multi-dimensional perception support for upper-level intelligent decision-making.